Who Reviews the AI?
Code Review with Codex

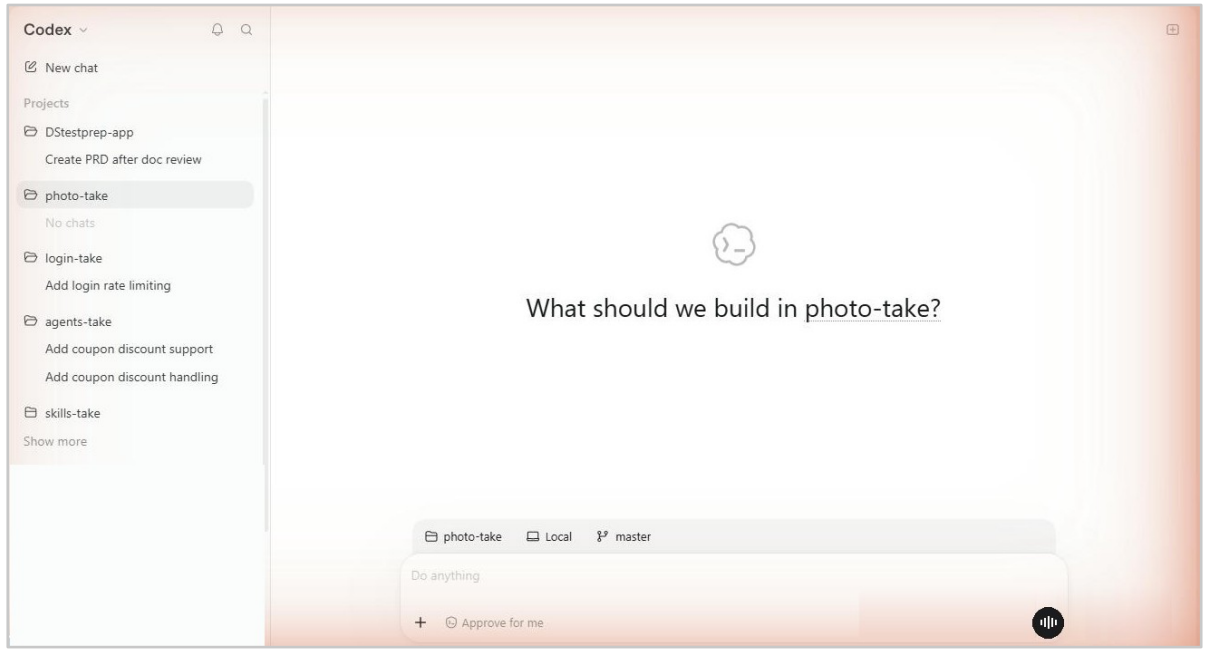


Akmal Khan
Senior Full Stack AI Architect

akmalkhaniub.github.io

Author Audition

Meet Codex: OpenAI’s coding agent



Reads your repo

Writes the change

Runs your tests

You review the diff

Give it a task in plain English. It works in your repo like a teammate.

Codex app · CLI · IDE extension. Today: the Codex app on Windows.

Author Audition

Overview



Codex writes the code in minutes.
Now reviewing is the slow part.

📝 Writing — 5 min

🔎 Reviewing — 30 min

WHAT'S IN IT FOR YOU

Let Codex do the first review, so you only check what matters.

1 Feature Dev → 2 Senior Reviewer → **3 The proof**

The PR Looks Green. Would You Approve It?

app/profile.py

```
def upload_avatar(user_id, file):
    destination = AVATAR_DIR /
f"{uuid4().hex}.png"
    with open(destination, "wb") as target:
        copyfileobj(file, target)
    return f"/avatars/{destination.name}"
```

💬 The task

Let users upload a profile photo.

✅ **9 tests passed**

✅ **Even handles errors**

🤔 Would you approve this PR?

Hold that thought.

Tests check what we asked for. Not what we forgot to ask.

What Happens If We Ship It?

Friday, 5 pm

✅

Tests are green.
PR approved.

Saturday, 2 am

⚠

A rare failure
hits the uploads.

Monday, 9 am

🔥

Broken profile photos.
Tickets pile up.

Monday, 10 am

🙋

"Who approved
this PR?"

The fix takes minutes. A bug in production costs your users, and your weekend.

Today: catch it before you click Approve.

Author Audition

Every Pull Request Needs a Second Pair of Eyes



vs



Feature Developer

“Did I satisfy the ticket requirements?”

Senior PR Reviewer

“What breaks this in production?”

Codex wears both hats. Never in the same prompt.

Switch hats with /review

1

Type /review
in the composer

2

Pick what to review
Uncommitted = Codex’s new edits

3

Read the findings
On the lines, most important first

4

You decide
Codex changes nothing yet

What can it review?

Uncommitted changes

↑ today

Against a base branch

Point at the line. Keep the fix small.

```
def upload_avatar(user_id, file):

    ...

+       with destination.open("xb") as target:
```

💬 **Your comment**
If saving fails, delete the half-saved file.

You send:
“Address the inline comments and keep the scope minimal.”

Codex: 1 fix + 1 test.
Nothing else touched.

Build → **Review** → **Commit**

Never jump straight from Build to Commit.

Demo



First the Feature Dev, then the Senior Reviewer.
Same Codex, same app.

1. Ask Codex to add profile photo upload (Feature Dev)
2. /review → uncommitted changes (Senior Reviewer)
3. **Comment on the risky line**
4. **Small fix, then review again**

One review. One real risk caught.

Feature Dev only	Feature Dev + Senior Reviewer
✓ Photo upload works	✓ Photo upload works
✓ 9 tests pass	✓ Tests pass, plus 1 new test
✗ Disk full? A broken file is left behind	✓ Failed saves are cleaned up

The tests were green both times. The review made the difference.

Your turn: one habit

`/review`

After every Codex task. Before every commit.

- 📁 **What if it fails halfway?**
- 🔑 **Who is allowed to do this?**
- 💥 **What if the input is empty or huge?**

Questions a senior reviewer asks. Check that the findings cover them.

Summary



Review is the new bottleneck

Switch hats: /review before commit

Point at the line. Keep the fix small

Codex finds. You decide